scx_hush: Unfair-by-Design CPU Scheduling for Mixed Latency-Critical and Background Workloads with `sched_ext`

Mohak

Technical report, October 23, 2026

Abstract

General-purpose operating systems schedule CPU time by proportional share. Lowering the priority of background work (nice values, `SCHED_IDLE`, cgroup `cpu.idle`) shrinks its share but never removes it, so deferrable jobs such as compilation, compression or database maintenance still delay latency-critical foreground work. Strict prioritisation removes that interference but brings two new failure modes: unbounded starvation and priority inversion through shared locks. We present *scx_hush*, a CPU scheduler for Linux, loaded at runtime through the extensible scheduler class `sched_ext`. It gives foreground work strict precedence and preempts background work as soon as foreground work wakes. A rate-limited starvation guard bounds how long background work waits. Priority inversion is resolved with four detection tiers (application hints, PI futexes, glibc mutexes and uninterruptible-sleep escape) plus a same-process fallback. Classification comes from cgroups, process names and user IDs in a policy file, so unmodified applications benefit. scx_hush generalises the database-specific UFS design [2] to any machine that mixes the two kinds of work. This report describes the design, a complete implementation (about 6,100 lines of BPF, C and Python, excluding tests), its verification, and an evaluation methodology with nine experiments. Baseline measurements on a 4-vCPU machine confirm the interference the design targets: under CPU-bound background load, the p99 wakeup latency of a foreground benchmark rose about 49× with the default scheduler, and neither nice 19 nor `SCHED_IDLE` reduced it. The in-kernel evaluation of scx_hush itself requires a `sched_ext`-enabled kernel and is left as the immediate next step; no scx_hush performance figures are claimed here.

1 Introduction

Many machines run two kinds of work at once. One kind is something a person or a control loop is waiting on: a video call, an editor, a robot's 100 Hz control loop, the query threads of a database. The other kind is deferrable bulk work: a parallel compiler build, log compression, a backup, `VACUUM`. Table 1 lists representative settings.

Problem statement. Linux offers several priority mechanisms: nice values, the `SCHED_IDLE` policy and cgroup CPU weights including `cpu.idle`. All of them are proportional-share mechanisms. A background task at the lowest weight still receives slices. It still occupies a CPU when a foreground task wakes, so the foreground task waits for the next scheduling decision. And it can still hold a lock that a foreground task needs. The UFS authors show that OS schedulers, despite supporting priorities, do not reliably isolate high-priority tasks, and remain vulnerable to priority inversion [2]. The problem this work addresses is:

> *How can a general-purpose machine run deferrable background work only on CPU time that latency-critical foreground work does not want, without modifying appli-*

Table 1: The two-class pattern on different machines.

Setting	Foreground (latency-critical)	Background (deferrable)	Symptom today
Developer laptop	compositor, editor, video call	compiler builds, indexing, backups	UI stutter, dropped frames
Robot / drone computer	perception, planning, control loop	log compression, map building, upload	control-loop jitter
Database server	query and request threads	vacuum, compaction, analytics	tail latency doubles in maintenance windows
Shared lab / CI machine	interactive SSH, notebooks	long training or test jobs	interactive sessions unusable

cations, while bounding background starvation and resolving priority inversion, and with no risk to system stability?

Contributions.

1. A survey of proportional-share, colocation, user-space and pluggable scheduling approaches, and five research gaps that they leave (section 3).
2. The design of scx_hush: strict class ordering with immediate preemption, a watchdog-safe starvation guard, and a four-tier priority-inversion mechanism with bounded, capped boosts (section 4).
3. A complete implementation on `sched_ext`: the BPF scheduler, a C daemon and loader, a CLI, a hint library and an evaluation harness (section 5).
4. A reproducible evaluation methodology of nine experiments against six baselines and one ablation. We report verification results and baseline measurements, and state clearly which results are still outstanding (sections 6 and 7).

2 Background

Proportional-share scheduling in Linux. Linux's fair class orders runnable tasks by weighted virtual runtime. The Completely Fair Scheduler did this from 2.6.23, and EEVDF [18] has done so since 6.6. Nice values change a task's weight: nice 19 weighs 15, against 1024 at nice 0. `SCHED_IDLE` gives a task the minimum weight, and cgroup v2 `cpu.idle` [19] gives a whole group idle-class treatment. In all three cases, low priority means a small share, not "none while foreground is runnable".

sched_ext. The extensible scheduler class [20], merged in Linux 6.12, lets a BPF program implement `struct sched_ext_ops`. Tasks flow through *dispatch queues* (DSQs): one global FIFO, one local DSQ per CPU, and any number of custom DSQs, which can be FIFOs or virtual-time priority queues. A CPU runs the head of its local DSQ. When that is empty, the CPU calls `ops.dispatch()`. Safety is built in: a BPF error, a runnable task left unscheduled past a watchdog timeout (at most 30 s), or SysRq-S aborts the BPF scheduler and returns every task to the fair class. The kernel documentation warns that `sched_ext` kfuncs carry no ABI stability guarantee. For example, `scx_bpf_dispatch` has been renamed to `scx_bpf_dsq_insert`.

Priority inversion and futexes. Priority inversion occurs when a high-priority task waits for a resource held by a lower-priority task. Priority inheritance [17] temporarily raises the holder's priority until it releases the resource. User-space locks on Linux are built on futexes [5].

PI futexes (`FUTEX_LOCK_PI`) keep the owner's thread ID in the futex word as kernel ABI. Default glibc mutexes use `FUTEX_WAIT` and record the owner in a separate `__owner` field. Strict prioritisation makes inversion worse, not better: a low-priority holder may never run at all.

3 Literature Review

Proportional-share schedulers. Lottery [23] and stride [24] scheduling established proportional share as a flexible resource abstraction. EEVDF [18] added deadlines for accuracy, and it is now Linux's default. BVT [4] let latency-sensitive threads *borrow* virtual time to dispatch early, so it gives low latency inside a fair-share framework. All of these schedulers, however, keep giving low-weight work a share by design. Lozi et al. [12] showed further that Linux's general-purpose balancing logic can leave cores idle while work waits, and that scheduler correctness is hard to get right, which motivates making new policies safe to deploy.

Real-time scheduling. `SCHED_DEADLINE` [10] and PREEMPT_RT provide hard guarantees. They require per-task timing parameters and careful admission control, so they do not fit unmodified desktop or server applications. scx_hush targets soft real-time behaviour and leaves hard guarantees out of scope.

Colocation of latency-critical and best-effort work. Datacenter systems colocate latency-critical (LC) services with best-effort batch jobs to raise utilisation. Borg [22] separates production and batch priorities across a cluster. Heracles [11] and PARTIES [3] partition cores, cache, memory bandwidth and power using feedback control on LC latency. These systems work at coarse granularity: they adjust allocations over seconds, rather than deciding every context switch.

Microsecond-scale and user-space scheduling. Shinjuku [8] uses preemption at microsecond intervals to cut tail latency. Shenango [15] and Caladan [6] reallocate cores between LC and batch applications at microsecond timescales. These systems deliver excellent tail latency, but they need applications built on their runtimes or kernel-bypass stacks.

Pluggable kernel scheduling. ghOSt [7] delegates Linux scheduling decisions to a user-space agent. Syrup [9] lets applications supply scheduling policies, written in eBPF [14], across the stack. `sched_ext` [20, 16] is the upstream mechanism: the policy runs as verified BPF inside the kernel and falls back to the default scheduler on failure. Recent work applies `sched_ext` to resource control for AI agents [25] and to LLM-generated policies [26]. Bechberger [1] gives a practical introduction.

Lock-holder preemption. Descheduling a lock holder is a known pathology in virtualised systems. Uhlig et al. [21] studied lock-holder preemption in multiprocessor VMs and proposed making the hypervisor aware of locks held by guests. Classic priority inheritance [17] solves the problem for kernel-visible locks with known priorities.

UFS: the direct basis. UFS [2] restricts background database tasks to idle CPU capacity using an eBPF scheduler. It preempts them immediately when time-sensitive tasks arrive, and handles priority inversion with application hints passed through BPF maps. Integrated into PostgreSQL, UFS improved time-sensitive throughput by up to 2× and halved tail latency compared with existing Linux options [2]. It is evaluated on a single application whose code was modified to send hints.

Table 2: Comparison of approaches. ✓ = provided, ∘ = partial, — = not provided.

Approach	Strict FG precedence	Unmodified apps	Inversion handling	Bounded starvation	Safe fallback
nice / `SCHED_IDLE` / `cpu.idle`	—	✓	∘ (PI futex only)	✓	✓
BVT [4]	∘	✓	—	✓	—
Heracles / PARTIES [11, 3]	∘ (coarse)	✓	—	✓	—
Shinjuku / Shenango / Caladan	✓	—	—	∘	—
ghOSt [7]	policy-dependent	✓	—	policy-dependent	∘
UFS [2]	✓	— (hints)	✓(hints)	∘	✓
scx_hush (this work)	✓	✓	✓(4 tiers)	✓	✓

User space: applications (unmodified; optional `libhush` hints) · `hushd` (loads BPF, resolves policy, writes maps, reads stats) · `hushctl` and `policy.toml`

↓ futex calls, uprobes ↕ BPF maps (classes, config, stats, trace)

Kernel (`sched_ext`): hook programs (futex, exec, rename, uprobes) → scx_hush scheduler (`select_cpu`, `enqueue`, `dispatch`, `running`, `stopping`, `tick`, ...) → `FG_DSQ` (vtime, 5 ms) · `NORMAL_DSQ` (vtime, 10 ms) · `BG_DSQ` (FIFO, 20 ms) → each CPU drains FG, then NORMAL, then BG. The `sched_ext` core's watchdog and SysRq-S fall back to the default scheduler.

Figure 1: scx_hush architecture.

Research gaps. Table 2 summarises the field. We identify five gaps:

G1: Isolation without starvation.
Proportional-share mechanisms never give foreground strict precedence. Strict schemes leave background progress unbounded, or bounded only by an external watchdog.

G2: Generality.
Strict isolation with inversion handling has been shown for one modified application (UFS on PostgreSQL), not for unmodified general-purpose workloads.

G3: Inversion without application changes.
Hint-based inversion handling needs source changes. Kernel PI applies only to PI futexes, which default mutexes do not use.

G4: Safe deployment.
Kernel-patch schedulers risk system stability. Strict-priority policies can also trip the `sched_ext` watchdog if starvation is not bounded well below its timeout.

G5: Reproducible comparison.
Existing reports rarely compare strict isolation against all of Linux's existing options (nice, `SCHED_IDLE`, `cpu.idle`) on the same hardware, with an ablation of inversion handling.

4 Design

scx_hush splits into a *policy plane* in user space and a *decision plane* in the kernel. The two communicate only through BPF maps (fig. 1).

Algorithm 1 Starvation guard, run at the top of `dispatch` on CPU c.

for $q \in \{$`NORMAL_DSQ`, `BG_DSQ`$\}$ **do**
  $t \leftarrow \mathrm{head}(q)$; **if** $t = \perp$ **continue**
  $w \leftarrow now - t.runnable_at$
  $hard \leftarrow w > W/2$
  if $w > G$ **and** $(now - c.last_guard \geq G$ **or** $hard)$ **and** $c \in t.cpus$ **then**
    move t to the local DSQ of c; mark t GUARDED (exempt from preemption)
    $c.last_guard \leftarrow now$; **return**
  end if
end for
drain `FG_DSQ`, else `NORMAL_DSQ`, else `BG_DSQ`

4.1 Classes and dispatch

Every task belongs to exactly one class: FOREGROUND, NORMAL (unclassified tasks and kernel threads) or BACKGROUND. A task's *effective* class is computed in a single helper, so no two code paths can disagree about it:

$$\mathrm{eff}(t) = \begin{cases} \mathrm{FG} & \text{if } (boost_until(t) > now \ \vee \ crit_depth(t) > 0) \wedge cap_ok(t), \\ \mathrm{NORMAL} & \text{if } base(t) = \mathrm{BG} \text{ and a D-state escape window is open,} \\ base(t) & \text{otherwise.} \end{cases} \tag{1}$$

where $cap_ok(t)$ holds while the task's boosted time in the current second is under the 10% cap (section 4.3).

On wake-up, `select_cpu` places a task of any class directly on an idle CPU, because idle cycles are free and this keeps the scheduler work-conserving. Otherwise `enqueue` inserts the task into its class DSQ. FOREGROUND and NORMAL tasks are ordered by weighted virtual time: used runtime scaled by $100/w$, where w is the nice weight. On wake-up, a task's virtual time is clamped to no less than the class minimum minus one slice, so a long sleeper cannot monopolise the CPU. BACKGROUND tasks queue in FIFO order and get a longer slice, which reduces context switches.

Preemption. When a FOREGROUND or NORMAL task becomes runnable and no CPU is idle, `enqueue` scans the per-CPU contexts, starting at the task's previous CPU. It looks for a CPU whose current class is BACKGROUND, claims it with a compare-and-swap (so concurrent wakers do not all target the same CPU) and sends `SCX_KICK_PREEMPT`. The scan is bounded by the CPU count. A `tick` backstop sets the slice of a running BACKGROUND task to zero whenever higher-class work is queued; this covers the race in which a kick and a new background dispatch cross.

4.2 Starvation guard

Algorithm 1 runs at the top of `dispatch`. When the head of `BG_DSQ`, its oldest waiter, has waited longer than the guard interval G (2 s by default), the CPU grants it one slice. Each CPU may grant at most one guard slice per G. This bounds the cost to foreground work at s_{bg}/G, which is 20 ms in 2 s, or 1%. The guard also covers `NORMAL_DSQ`, so saturating foreground load cannot starve kernel threads.

The per-CPU rate limit alone is not watchdog-safe. With n background tasks on c CPUs, a FIFO tail waits roughly $(n/c) \cdot G$, which can exceed the watchdog timeout W, and the kernel would then eject the scheduler. A *hard guard* therefore ignores the rate limit once the head has waited $W/2$. The daemon refuses any configuration with $G > W/4$.

4.3 Priority inversion

Strict priority makes inversion worse: a background lock holder may never run under continuous foreground load. scx_hush therefore treats inversion as a core feature and detects it in four tiers, ordered from most precise to most general (table 3).

Table 3: Inversion detection tiers.

Tier	Mechanism	Precision	App changes
1 Hints	uprobes on `hush_critical_enter/exit` in `libhush.so`	exact	yes
2 PI futex	`FUTEX_LOCK_PI`: owner TID read from the futex word (kernel ABI)	exact	no
3 glibc mutex	`FUTEX_WAIT`: owner TID read from `__owner`, offset 8, validated by a self-test	heuristic	no
4 D-state escape	FG in uninterruptible sleep >10 ms opens a 20 ms window in which BG runs as NORMAL	coarse	no
Fallback	boost BG threads of the waiter's process that ran in the last 10 ms	over-boosts	no

A detected holder gets $boost_until = now + 50\,\text{ms}$. Hook programs cannot move tasks between DSQs, so they increment a `boost_pending` counter. The next `dispatch` on any CPU then walks the first 64 entries of `BG_DSQ` and pulls boosted tasks out. A boost ends on `FUTEX_WAKE` or `FUTEX_UNLOCK_PI` at the same address, on `hush_critical_exit`, or on expiry. It is renewed only while the waiter is still blocked on that address.

Two safeguards limit abuse. First, a task's boosted time is capped at 10% of each second, whatever the boost source, and refused hint calls are counted. Second, because the owner TID is read from user memory, a glibc owner must belong to the waiter's process, and a PI owner must share its process or its effective UID. Without that check, one user could lend FOREGROUND to another user's task.

4.4 Classification and safety

Classes come from ordered first-match rules on cgroup v2 paths (globs that also match descendants), executable or thread names, and UIDs. A cgroup match takes precedence over a name match. An unmatched child inherits its parent's class at fork, and kernel threads are always NORMAL. The daemon resolves cgroup rules in user space and stores one class per cgroup ID, so the BPF side does a single hash lookup. A generation counter, together with hooks on `exec` and `task_rename` and a cgroup-ID comparison at enqueue, triggers re-classification without reloading the BPF program.

For safety, every scheduling decision runs as verified BPF, and the `sched_ext` core reverts to the default scheduler on any error, watchdog stall, SysRq-S or daemon exit.

5 Implementation

Table 4 lists the components. All `sched_ext` API names are confined to one header, `hush_compat.bpf.h`, which wraps the `sched-ext/scx` compatibility headers [16] at a pinned commit. A kfunc rename or signature change, such as `scx_bpf_dsq_move_to_local` gaining an `enq_flags` argument, is therefore absorbed in one place. The BPF object is built with CO-RE against BTF from the target kernel; it uses the running kernel's BTF when that has `sched_ext` types, and the vendored BTF otherwise. One object covers x86-64 and another covers arm64.

The control socket accepts read-only commands from any user. Commands that change state require a root peer, which the daemon checks through `SO_PEERCRED`. Two modes exercise the

Table 4: Implementation components (lines of code).

Component	Responsibility	LOC
`bpf/scx_hush.bpf.c`	13 `sched_ext` callbacks; futex, exec, rename and uprobe hooks; 10 maps	1,221
`hushd/` (C, libbpf)	policy (TOML), validation, cgroup resolution with inotify, Unix-socket NDJSON control, self-test, trace, exit report	3,133
`cli/hushctl.py`	status, class, promote/demote/reset, reload, self-test, trace	210
`libhush`	empty probe-point functions for tier-1 hints	69
`tools/`	`inversion_micro` (E5) and `periodic_loop` (E2, E9) workloads	324
`bench/real/`	harness for B0–B6 and E1–E9, bootstrap CIs, figures	919

control plane without a `sched_ext` kernel. `-dry-run` prints the resolved policy, and `-no-load` runs the full daemon (socket, inotify, reload and overrides) without loading BPF.

6 Evaluation Methodology

The evaluation answers three questions. Does foreground stay isolated? Does background still progress? Does inversion handling matter? Tables 5 and 6 define the baselines and experiments.

Table 5: Baselines.

ID	Configuration
B0	Foreground alone (reference)
B1	Default scheduler (EEVDF), background at the same priority
B2	Background at nice 19
B3	Background under `SCHED_IDLE`
B4	Background in a cgroup with `cpu.idle`=1
B5	scx_hush with inversion handling off (ablation)
B6	scx_hush, full

Table 6: Experiments.

ID	Workload	Primary metric	Configs
E1	schbench [13] under a parallel build or CPU stress	p99, p99.9 wakeup latency	B0–B6
E2	100 Hz loop with `zstd -19` and `rsync`	p99 and max period deviation	B0–B6
E3	pgbench with `VACUUM FULL` and an analytic scan	throughput, p99 transaction latency	B1, B3, B6
E4	E1/E2 background	background CPU-seconds per minute	B1–B6
E5	`inversion_micro`: PI, glibc and hinted mutexes	blocked time per acquisition	B5, B6
E6	E1, guard $\in \{0.5, 1, 2, 4\}$ s	FG p99 vs longest BG wait	B6
E7	hackbench and schbench, no background	overhead vs default	B1, B6
E8	all scenarios cycled for 24 h	watchdog aborts	B6
E9	E2 on Raspberry Pi 5 / Jetson	p99 jitter	B1, B3, B6

Foreground and background run in separate cgroups. The `cpu` controller is enabled only for B4. Background progress is measured as the CPU time consumed by the background cgroup during the measurement window (cgroup `cpu.stat`), which works for any background command. Each configuration runs r times after a warm-up. We report the median and a 95% bootstrap confidence interval, computed with a fixed seed. Each campaign records the policy used, `uname -a`, the git commit and the CPU governor.

7 Results

7.1 Implementation verification

Table 7 summarises the verification performed on the implementation. The development machine's kernel (Linux 6.18) was built without `CONFIG_SCHED_CLASS_EXT`. On that kernel the daemon refuses to load and says why, so verifier acceptance and in-kernel behaviour remain to be tested.

Table 7: Verification results.

Check	Scope	Result
BPF build	CO-RE object and skeleton, x86-64 and arm64, clang 18	pass, 0 warnings
User-space build	`hushd`, `libhush`, tools; `-Wall -Wextra`	pass, 0 warnings
C unit tests	TOML parsing, FR-12 validation, glob/ancestor matching, first-match order, overrides	48/48
Python tests	control protocol, `hushd -no-load` over its socket, reload accept/reject, SIGHUP, inotify, malformed input; simulator	32/32
Self-test	PI futex word carries owner TID; glibc 2.39 `__owner` at offset 8	pass
Privilege	unprivileged `status` allowed, `promote` refused	pass
Kernel load	verifier acceptance, `sched_ext` state	*pending*

7.2 Baseline measurements

We ran E1 and E2 for B0–B3 on a 4-vCPU virtual machine (Intel Xeon, 2.1 GHz) with 3 repetitions of 20 s after 5 s of warm-up. The background load was `stress-ng` on all CPUs for E1, and `zstd -19` with `rsync` for E2. B4 could not run because the VM's hybrid cgroup hierarchy does not expose the `cpu` controller to cgroup v2. B5 and B6 could not run because the kernel lacks `sched_ext`.

Table 8: E1: schbench wakeup latency (μs) and background progress. Median over repetitions, 95% bootstrap CI in brackets.

Config	p99 wakeup	p99.9 wakeup	BG CPU-s/min
B0 FG alone	126 [120, 148]	2676 [1610, 2732]	0 [0, 0]
B1 EEVDF	6120 [6008, 6136]	8272 [8104, 9520]	79 [79, 85]
B2 nice 19	6152 [6072, 6168]	9040 [8976, 9264]	84 [82, 84]
B3 SCHED_IDLE	6616 [6408, 8272]	10000 [9328, 10640]	90 [81, 95]
B4 cpu.idle	n/a	n/a	n/a
B5 scx_hush (no inv.)	*pending*	*pending*	*pending*
B6 scx_hush	*pending*	*pending*	*pending*

Observations. With CPU-bound background load at the same priority (B1), the p99 wakeup latency of schbench rose to 48.6× the foreground-alone value (B0). Background at nice 19 (B2) gave 48.8× and `SCHED_IDLE` (B3) 52.5× (table 8). E2 is inconclusive on this machine. The 100 Hz loop running *alone* (B0) already showed a p99 period deviation of 9.4 ms and a maximum of 66 ms (table 9), i.e. the virtual machine's timer and vCPU scheduling dominate the jitter, so differences between B1–B3 cannot be attributed to the background priority mechanism. E2 must be repeated on bare metal. Background progress was similar across B1–B3 (79–90 CPU-seconds per minute in E1 on 4 vCPUs). The question scx_hush targets is whether strict precedence can bring foreground latency back to B0 at a similar background cost. These measurements

Table 9: E2: 100 Hz control loop, period deviation and wake-up lateness (ms). Median over repetitions, 95% bootstrap CI in brackets.

Config	p99 |dev|	max |dev|	p99 late	BG CPU-s/min
B0 FG alone	9.38 [8.73, 9.47]	66.4 [55.8, 67.7]	10.09 [3.88, 14.34]	0 [0, 0]
B1 EEVDF	9.48 [7.31, 9.48]	82.1 [52.9, 96.8]	25.40 [4.46, 32.19]	227 [223, 228]
B2 nice 19	9.48 [9.47, 9.48]	81.0 [70.4, 99.1]	17.15 [9.26, 24.50]	224 [220, 228]
B3 SCHED_IDLE	2.48 [2.34, 3.98]	10.2 [8.3, 14.9]	2.21 [2.08, 2.38]	221 [221, 223]
B4 cpu.idle	n/a	n/a	n/a	n/a
B5 scx_hush (no inv.)	*pending*	*pending*	*pending*	*pending*
B6 scx_hush	*pending*	*pending*	*pending*	*pending*

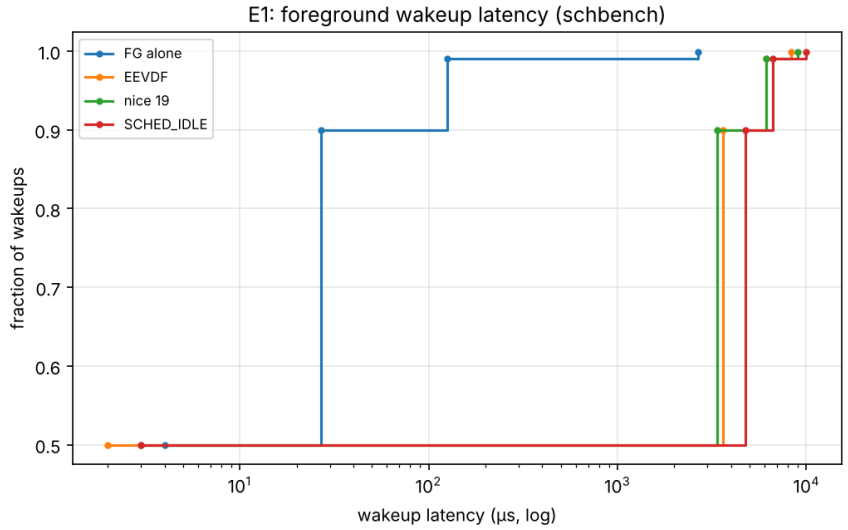


Figure 2: E1: schbench wakeup-latency distribution per baseline, from schbench's percentile points (median over repetitions). Background load moves the tail from about 0.1 ms to about 6 ms, and lowering its priority does not move it back.

quantify the interference that motivates scx_hush on this machine; they are not an evaluation of scx_hush, and the VM's noise (section 9) means only large effects should be read from them.

7.3 scx_hush results

E1–E9 for B5 and B6 are *pending*. They need a kernel with `CONFIG_SCHED_CLASS_EXT=y`. The PRD's targets, against which those results will be judged, are:

- foreground p99 latency within 1.2× of foreground alone, and throughput at least 90% of foreground alone;
- background using at least 80% of the idle CPU share;
- no background wait longer than the guard interval;
- a lock holder blocking foreground for at most one boost slice after detection;
- under 2% overhead with no background work;
- zero watchdog aborts in a 24-hour soak.

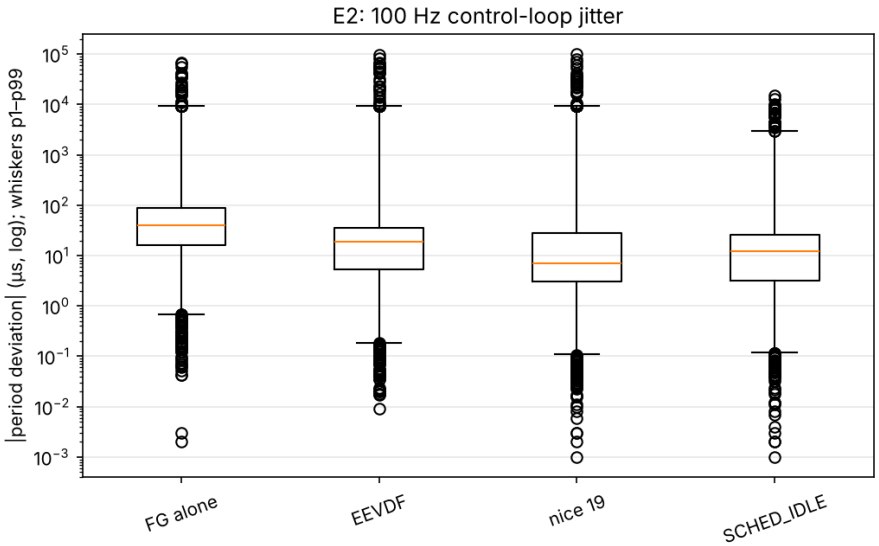


Figure 3: E2: distribution of |period deviation| of a 100 Hz loop under each baseline (all repetitions pooled). The foreground-alone reference is itself noisy on this virtual machine, so this experiment is inconclusive here.

8 Gaps Fulfilled

Table 10 maps each research gap from section 3 to the mechanisms that address it, and states how the claim is or will be supported.

Table 10: How scx_hush addresses the identified gaps.

Gap	Mechanism in scx_hush	Evidence
G1	Strict DSQ order, preemption kick plus tick backstop; per-CPU rate-limited guard with a watchdog-safe hard guard	design; E1, E4, E6 *pending*
G2	Policy-driven classes (cgroup, comm, uid), fork inheritance, exec/rename re-classification; no application changes	implemented, tested
G3	PI-futex and glibc-mutex owner detection in the kernel; hints optional; same-process fallback	implemented; E5 *pending*
G4	All logic in verified BPF; `sched_ext` fallback; guard $\leq W/4$ enforced; API isolation (NFR-9)	implemented; E8 *pending*
G5	One harness for B0–B6 and E1–E9 with raw CSV, metadata and bootstrap CIs	implemented; baselines measured

9 Discussion and Limitations

Threats to validity. The baselines were measured on a shared virtual machine, where hypervisor scheduling adds noise that bare metal would not have. Short runs (20 s) and three repetitions give wide confidence intervals. On this VM, the foreground-alone 100 Hz loop (E2, B0) already missed deadlines by up to tens of milliseconds, so jitter experiments can only be judged on bare metal. The final campaign should use bare metal, pinned frequency, five repetitions of 120 s, and a real kernel build as background load, as the methodology specifies.

Known limitations.

- Locks shared across processes through shared memory are detected only by tier 1.

- Spinlocks and lock-free retry loops are invisible to every tier; only the guard protects against them.
- Tier 3 depends on glibc's mutex layout and is disabled when the self-test fails.
- Owner lookup uses the initial PID namespace, which limits tiers 2–3 inside containers.
- A single global `BG_DSQ` may contend beyond 16 CPUs; E7 will tell whether it needs sharding.

10 Conclusion and Future Work

scx_hush gives general-purpose Linux machines strict foreground isolation with bounded background starvation and in-kernel priority-inversion handling, without application changes. It is deployed safely through `sched_ext`. We contributed a gap analysis, a design that closes a watchdog-safety hole left by per-CPU rate limiting alone, a complete implementation, and a reproducible evaluation methodology. Baseline measurements (E1) show that the existing Linux priority mechanisms leave a roughly 49× tail-latency penalty under background load, which is the interference scx_hush is designed to remove.

The immediate next step is to run E1–E9 on a `sched_ext`-enabled kernel on bare metal and on embedded boards. That will test verifier acceptance and the PRD's quantitative targets, and answer the open design questions: whether NORMAL should also be preemptible by FOREGROUND, whether `BG_DSQ` needs sharding, and whether kick latency stays below 100 μs.

References

[1] Johannes Bechberger. Writing a minimal scheduler with eBPF, sched_ext, and C. FOSDEM 2025, 2025.

[2] Bilodeau-Savaria, Nidzwetzki, Scherzinger, and Kemme. Unfair by design: eBPF-based scheduling of mixed database workloads. arXiv:2605.02377, May 2026.

[3] Shuang Chen, Christina Delimitrou, and José F. Martínez. PARTIES: QoS-aware resource partitioning for multiple interactive services. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[4] Kenneth J. Duda and David R. Cheriton. Borrowed-virtual-time (BVT) scheduling: Supporting latency-sensitive threads in a general-purpose scheduler. In *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP)*, 1999.

[5] Hubertus Franke, Rusty Russell, and Matthew Kirkwood. Fuss, futexes and furwocks: Fast userlevel locking in Linux. In *Proceedings of the Ottawa Linux Symposium*, 2002.

[6] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. Caladan: Mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[7] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, and Christos Kozyrakis. ghOSt: Fast & flexible user-space delegation of Linux scheduling. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021.

[8] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for μsecond-scale tail latency. In

Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2019.

[9] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. Syrup: User-defined scheduling across the stack. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021.

[10] Juri Lelli, Claudio Scordino, Luca Abeni, and Dario Faggioli. Deadline scheduling in the Linux kernel. *Software: Practice and Experience*, 46(6), 2016.

[11] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd International Symposium on Computer Architecture (ISCA)*, 2015.

[12] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. The Linux scheduler: A decade of wasted cores. In *Proceedings of the 11th European Conference on Computer Systems (EuroSys)*, 2016.

[13] Chris Mason. schbench: a scheduler wakeup-latency benchmark. `https://github.com/masoncl/schbench`. Commit 24e32b8.

[14] Steven McCanne and Van Jacobson. The BSD packet filter: A new architecture for user-level packet capture. In *Proceedings of the USENIX Winter 1993 Conference*, 1993.

[15] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.

[16] sched_ext project. sched_ext schedulers and tools. `https://github.com/sched-ext/scx`. Commit d65d9eb, Oct. 2026.

[17] Lui Sha, Ragunathan Rajkumar, and John P. Lehoczky. Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, 39(9):1175–1185, 1990.

[18] Ion Stoica and Hussein Abdel-Wahab. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation. Technical Report TR-95-22, Old Dominion University, 1995.

[19] The Linux Kernel Documentation. Control group v2. `https://docs.kernel.org/admin-guide/cgroup-v2.html`. Accessed Oct. 2026.

[20] The Linux Kernel Documentation. Extensible scheduler class (sched_ext). `https://docs.kernel.org/scheduler/sched-ext.html`. Accessed Oct. 2026.

[21] Volkmar Uhlig, Joshua LeVasseur, Espen Skoglund, and Uwe Dannowski. Towards scalable multiprocessor virtual machines. In *Proceedings of the 3rd Virtual Machine Research and Technology Symposium (VM)*, 2004.

[22] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the 10th European Conference on Computer Systems (EuroSys)*, 2015.

[23] Carl A. Waldspurger and William E. Weihl. Lottery scheduling: Flexible proportional-share resource management. In *Proceedings of the 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1994.

[24] Carl A. Waldspurger and William E. Weihl. Stride scheduling: Deterministic proportional-share resource management. Technical Report MIT/LCS/TM-528, MIT Laboratory for Computer Science, 1995.

[25] Zheng et al. AgentCgroup: Understanding and controlling OS resources of AI agents. arXiv:2602.09345, 2026.

[26] Zheng et al. Towards agentic OS: An LLM agent framework for Linux schedulers, 2026.